

MongoDB Aggregation

Pipeline Queries — Structure, Samples & Explanations

April 2026

Collection Schema

All four queries run against a single **orders** collection. Each document records one customer order and embeds the line-items as an array.

```
// orders collection
{
  _id : ObjectId,
  customer_id : "C001",
  status : "delivered" | "pending" | "cancelled",
  order_date: ISODate("2025-03-15"),
  items: [
    { prod_id: "P101", name: "Laptop", price: 500 },
    { prod_id: "P205", name: "Mouse", price: 80 }
  ]
}
```

Sample Data Insert

Three representative documents are inserted so every query produces a traceable, deterministic result.

```
db.orders.insertMany([
  {
    customer_id: "C001", status: "delivered",
    order_date: ISODate("2025-03-15"),
    items: [{ prod_id:"P101", name:"Laptop", price:250 },
    { prod_id:"P205", name:"Mouse", price:80 }]
  },
  {
    customer_id: "C002", status: "pending",
    order_date: ISODate("2025-06-20"),
    items: [{ prod_id:"P101", name:"Laptop", price:500 },
    { prod_id:"P300", name:"Keyboard", price:120 }]
  },
  {
    customer_id: "C001", status: "delivered",
    order_date: ISODate("2025-11-10"),
    items: [{ prod_id:"P101", name:"Laptop", price:500 },
    { prod_id:"P205", name:"Mouse", price:80 },
    { prod_id:"P300", name:"Keyboard", price:250 }]
  }
]);
```

Query 01

Total Revenue from Delivered Orders

Sum all item prices where status = 'delivered'

WHAT

Compute the grand total of every item price across all delivered orders.

WHY

\$match early to shrink the working set before any expensive operations.

HOW

\$unwind explodes the items array; \$group with _id:null collapses everything into one sum.

Pipeline

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $unwind: "$items" },
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$items.price" }
    }
  }
]);
```

Expected Output

Orders 1 and 3 are delivered. Their item prices sum to **250 + 80 + 500 + 80 + 250 = 1 160**.

```
// Result
{ "_id": null, "totalRevenue": 1160 }
```

Query 02

Top 3 Spenders in 2025

Rank customers by total spend within a calendar year

WHAT

Find the three customers who spent the most money in 2025.

WHY

Date-range \$match uses ISODate objects; \$sort before \$limit ensures correct ranking.

HOW

\$unwind + \$group per customer → \$sort descending → \$limit 3.

Pipeline

```
db.orders.aggregate([
  { $match: {
    order_date: { $gte: ISODate("2025-01-01"),
    $lt: ISODate("2026-01-01") }
  }},
  { $unwind: "$items" },
  {
    $group: {
      _id: "$customer_id",
      totalSpent: { $sum: "$items.price" }
    }
  },
  { $sort: { totalSpent: -1 } },
  { $limit: 3 }
]);
```

Expected Output

C001 spent 1 160 (two delivered orders). C002 spent 620 (one pending order — \$match does not filter by status here, only by date). Both fall within 2025.

```
// Results (descending)
{ "_id": "C001", "totalSpent": 1160 }
{ "_id": "C002", "totalSpent": 620 }
```

Query 03

Quantity Sold per Product

Count how many times each product appeared across all orders

WHAT

Determine total unit sales for every product in the catalogue.

WHY

items is an array — \$unwind is mandatory to access individual products before grouping.

HOW

\$unwind → \$group by prod_id with \$sum: 1 counts appearances. Use \$sum: '\$qty' if a qty field exists.

Pipeline

```
db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.prod_id",
      quantitySold: { $sum: 1 }
    }
  }
]);
```

Expected Output

P101 (Laptop) appears in all three orders → qty 3. P205 (Mouse) appears in orders 1 and 3 → qty 2. P300 (Keyboard) appears in orders 2 and 3 → qty 2.

```
{ "_id": "P101", "quantitySold": 3 }
{ "_id": "P205", "quantitySold": 2 }
{ "_id": "P300", "quantitySold": 2 }
```

Query 04

Average Order Value per Month

Two-stage grouping — per-order total first, then monthly average

WHAT

Show the average total order value broken down by calendar month.

WHY

A single \$group would average per item, not per order — a classic mistake. Two stages are required.

HOW

Stage 1: \$group by orderId → per-order sum. Stage 2: \$group by month/year → \$avg of those sums.

Pipeline

```
db.orders.aggregate([
  { $unwind: "$items" },
  { // Stage 1 — per-order total
    $group: {
      _id: "$_id",
      orderTotal: { $sum: "$items.price" },
      order_date: { $first: "$order_date" }
    }
  },
  { // Stage 2 — monthly average
    $group: {
      _id: {
        year: { $year: "$order_date" },
        month: { $month: "$order_date" }
      },
      avgOrderValue: { $avg: "$orderTotal" }
    }
  },
  { $sort: { "_id.year": 1, "_id.month": 1 } }
]);
```

Expected Output

March 2025 has one order worth 330 → avg 330. June 2025 has one order worth 620 → avg 620. November 2025 has one order worth 830 → avg 830.

```
{ "_id": { "year": 2025, "month": 3 }, "avgOrderValue": 330 }
{ "_id": { "year": 2025, "month": 6 }, "avgOrderValue": 620 }
{ "_id": { "year": 2025, "month": 11 }, "avgOrderValue": 830 }
```

Pipeline Operator Quick Reference

Operator	Stage / Type	Purpose
<code>\$match</code>	Stage	Filter documents — always place early to reduce working set
<code>\$unwind</code>	Stage	Deconstruct an array field into one document per element
<code>\$group</code>	Stage	Aggregate documents; requires <code>_id</code> expression
<code>\$sort</code>	Stage	Order documents — always before <code>\$limit</code> in ranking queries
<code>\$limit</code>	Stage	Keep only the first N documents
<code>\$sum</code>	Accumulator	Total numeric values; <code>\$sum:1</code> counts occurrences
<code>\$avg</code>	Accumulator	Arithmetic mean of numeric values
<code>\$first</code>	Accumulator	Retain first value seen in group (use after <code>\$sort</code> if order matters)
<code>\$year / \$month</code>	Date Op.	Extract year or month integer from an ISODate field

Key Rules to Remember

- **`$match` first** — always filter early to reduce the number of documents flowing through the rest of the pipeline.
- **`$unwind` before `$group` on arrays** — you cannot aggregate inside a nested array without first flattening it.
- **`$sort` before `$limit`** — reversing the order gives incorrect top-N results.
- **Two `$group` stages for averages over computed values** — one stage to compute the per-document total, a second to average those totals.
- **ISODate for date comparisons** — MongoDB stores dates as BSON Date objects; plain string comparison does not work correctly.

All queries tested against MongoDB 7.x · Sample data produces fully deterministic outputs